

Programming Using C++ Level 1

Arrays with functions



Programming Using C++

Level 1

Arrays with functions

Definition

- ◆ Arrays can be passed to functions so that functions can work with multiple values.
- ◆ This allows a program to separate different tasks into different functions.
- ◆ In this program, one function reads the grades and another function calculates their average.

Explanation

- ◆ The program creates an array `Grades` with 3 float elements.
- ◆ `ReadGrades()` receives the array and stores the user's input in it.
- ◆ `CalculateAvgGrades()` receives the same array and uses its elements to calculate the average.
- ◆ The array is passed to the functions by writing its name when calling the function.

Steps in main()

1. Declare the array

- ◆ Create Grades with space for 3 float values.

2. Create ReadGrades()

- ◆ Receive the Grades array as a function parameter.

3. Store the grades

- ◆ Read the three values and store them in Grades[0], Grades[1], and Grades[2].

4. Create CalculateAvgGrades()

- ◆ Receive the Grades array as a function parameter.

5. Calculate the average

- ◆ Add the three grades and divide the result by 3.

6. Call ReadGrades()

- ◆ Pass Grades to ReadGrades() so the function can fill the array.

7. Call CalculateAvgGrades()

- ◆ Pass Grades to CalculateAvgGrades() and display the returned average.

💡 Example

◆ Array:

float Grades[3];

◆ Pass the array to a function:

ReadGrades(Grades);

◆ Calculate the average:

CalculateAvgGrades(Grades);

◆ If the grades are 80, 90, and 70, the function returns 80.

➡ Result

◆ If the user enters:

Grade1 = 80

Grade2 = 90

Grade3 = 70

◆ The program displays:

The average of grade is: 80

Note

- ◆ When an array is passed to a function, you pass its name without brackets containing the indexes.
- ◆ `ReadGrades(Grades)` passes the array to `ReadGrades()`.
- ◆ `CalculateAvgGrades(Grades)` passes the same array to `CalculateAvgGrades()`.
- ◆ The parameter `float Grades[3]` indicates that the function expects an array of float values with 3 elements.

⊘ Warning

- ◆ The array size and the indexes used by the function must match the intended number of elements.



Comparison

▨ Using Functions with Arrays

- ✓ Separates input and calculation into different functions.
- ✓ Makes the code easier to organize and understand.
- ✓ Allows the same functions to be reused with arrays.

▤ All Code Inside main()

- ✓ Input and calculation are written together.
- ✓ The main function becomes larger.
- ✓ Code organization and reuse become harder.

✦✦ Summary

- ✓ Arrays can be passed to functions to work with multiple values.
- ✓ Different functions can perform different operations on the same array.
- ✓ Combining arrays with functions makes programs more organized and reusable.

Thank you 😊